

packages/graph-core/src/plugins/tree-sitter-plugin.ts

Kind: added · Baseline: 8e0dc031dcfc · Target: NovaDiff · Generated: 2026-05-19T20:37:59.518Z

Overview

A new `TreeSitterPlugin` is added at `packages/graph-core/src/plugins/tree-sitter-plugin.ts` (lines 1-301).

It implements `AnalyzerPlugin` and offers structural analysis, import resolution, and call-graph extraction for languages whose grammars are loaded via `web-tree-sitter`.

Key changes

- **Imports & runtime setup** – Lines 1-12 add `createRequire` (node:module) and `dirname, resolve, extname` (node:path) to resolve `.wasm` files with `require.resolve`.
- **Config-driven language support** – The constructor (lines 57-98) accepts `LanguageConfig[]`; it builds `this.languages` and `_extensionToLang`. If no configs are supplied, it falls back to TypeScript and JavaScript (lines 75-84).
- **Extractor registration** – `registerExtractor` (lines 100-104) stores `LanguageExtractor`s keyed by language ID; the constructor registers either a supplied list or all `builtinExtractors` (lines 88-97). `genericExtractor` is used when no specific extractor exists.
- **Async `init()`** – Lines 125-200 load `web-tree-sitter`, initialise the parser, and asynchronously load each grammar from the configs (or legacy TS/JS grammars). Missing grammars are logged with `console.debug`.
- **Synchronous parsing** – `getParser` (lines 207-223) creates a parser for a file path after `init()` has run; it throws if called prematurely.
- **Analysis methods** – `analyzeFile`, `resolveImports`, and `extractCallGraph` (lines 225-301) use the pre-loaded parser and the appropriate extractor.
- **Graceful degradation** – If a grammar or extractor is missing, the plugin returns empty analysis results instead of throwing.

Impact

- **Correctness** – Analysis runs only after `init()`; missing grammars are logged, not fatal.
- **Maintainability** – Centralised language configuration and extractor registration simplify adding new languages.
- **Performance** – One-time async load of all grammars; subsequent analyses are synchronous and fast.
- **Compatibility** – Requires Node ≥ 12 for `createRequire`; uses ESM-only `web-tree-sitter`, so the plugin must be imported as ESM.

Risks & follow-ups

- **init() must be awaited** – Verify callers always await `init()` before any analysis; otherwise `getParser()` throws.
- **WASM resolution** – `require.resolve` may fail if package layout changes; add unit tests for path resolution.
- **Memory leaks** – Parsers and trees are deleted after use; ensure no stray references remain.
- **Legacy fallback** – Hard-coded TS/JS wasm paths may become stale; confirm they still exist in the dependency tree.